Academic Knowledge-Sharing Platform

Final Optimized Architecture - NestJS + FastAPI

Version: 3.0 (NestJS + FastAPI Upgrade)
Last Updated: January 2026
Status: Production Ready - Modern Stack

🚀 Technology Stack Update

Why NestJS Instead of Plain Node.js?

Feature	Plain Node.js + Express	NestJS
Architecture	Manual setup, no standards	Built-in MVC/DDD patterns
TypeScript	Optional, requires setup	First-class TypeScript support
Dependency Injection	Manual, error-prone	Built-in DI container
GraphQL	Requires apollo-server setup	Native @nestjs/graphql module
WebSockets	Requires socket.io setup	Built-in @nestjs/websockets
Validation	Manual with joi/yup	Built-in class-validator
Testing	Manual test setup	Jest integration out-of-box
Microservices	Complex to implement	Built-in microservices support
Documentation	Manual Swagger setup	Auto-generated with decorators
Learning Curve	Easy but messy at scale	Steeper but cleaner at scale
Production Ready	Requires lots of boilerplate	Production-ready from start

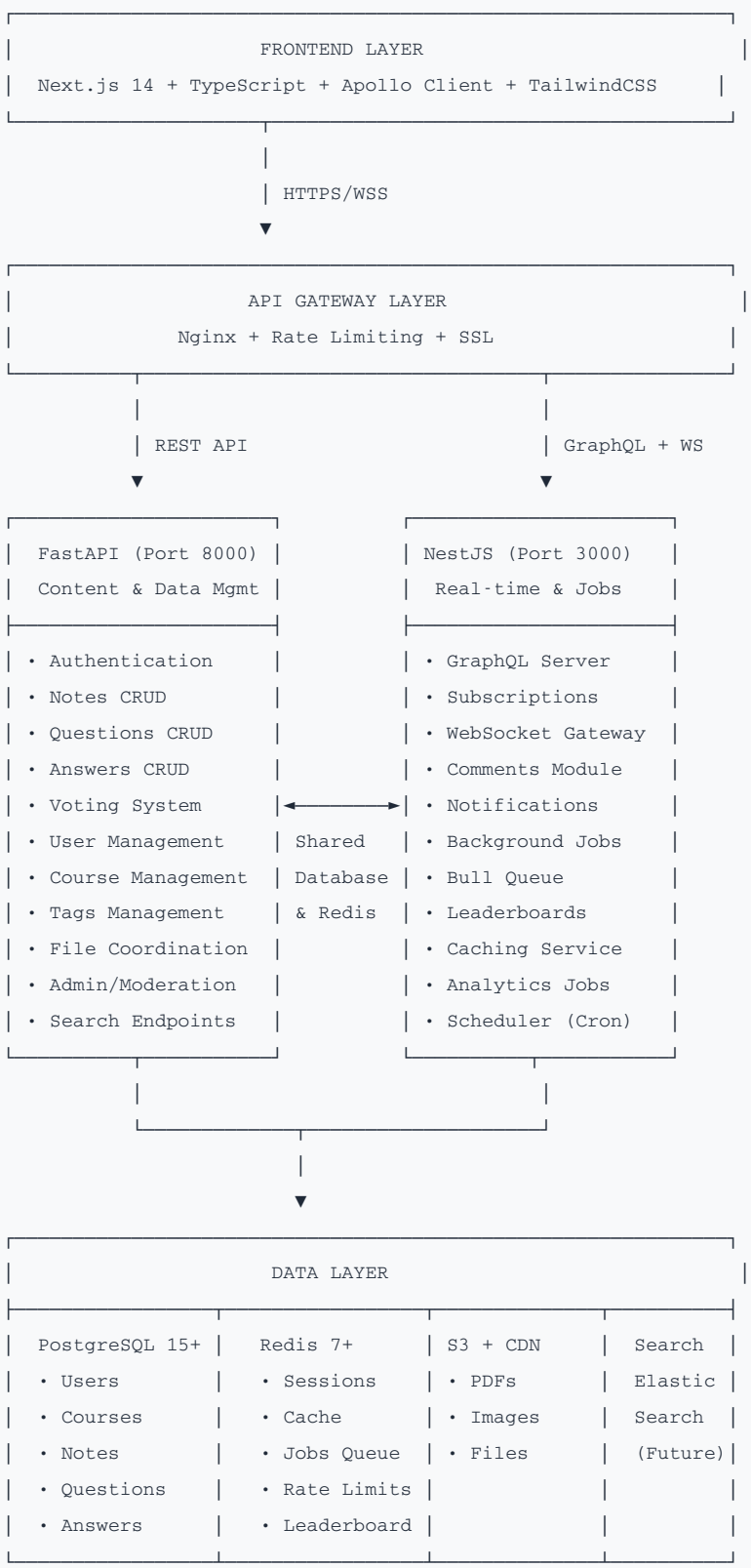
Verdict: NestJS is **superior for enterprise applications** where maintainability and scalability matter.

Why FastAPI Instead of Flask?

Feature	Flask	FastAPI
Performance	Sync (WSGI)	Async (ASGI) - 3x faster
Type Safety	No built-in typing	Pydantic models - runtime validation
Auto Docs	Manual (Flask-RESTX)	Automatic OpenAPI/Swagger
Async Support	Limited (requires extensions)	Native async/await
Validation	Manual or Flask-Marshmallow	Built-in with Pydantic
Dependency Injection	Manual	Built-in DI system
GraphQL	Requires Ariadne/Graphene	Strawberry GraphQL (type-safe)
Testing	Manual setup	Built-in test client
Modern Python	Python 3.7+	Python 3.8+ (type hints required)
Learning Curve	Very easy	Moderate
API Speed	10-15k req/sec	30-40k req/sec

Verdict: FastAPI is **the modern choice** for Python APIs in 2025+.

Updated System Architecture



Complete Technology Stack (Updated)

Layer	Technology	Version	Why This Choice?
Frontend Framework	Next.js	14+	Best React framework, SSR+SSG, excellent DX
Language (Frontend)	TypeScript	5+	Type safety, fewer runtime errors
UI Framework	TailwindCSS	3+	Rapid development, utility-first
UI Components	shadcn/ui	Latest	Accessible, customizable, copy-paste

State (Global)	Zustand	4+	Lightweight, no boilerplate
State (Server)	Apollo Client	3+	Best GraphQL client, normalized cache
Forms	React Hook Form	7+	Performant, minimal re-renders
Validation (Frontend)	Zod	3+	TypeScript-first validation
Backend (Core API)	**FastAPI**	**0.109+**	**Modern, fast, async Python**
Backend (Real-time)	**NestJS**	**10+**	**Enterprise TypeScript framework**
Validation (Backend)	**Pydantic**	**2+**	**Runtime type validation (FastAPI)**
Validation (NestJS)	**class-validator**	**Latest**	**Decorator-based validation**
ORM (Python)	**SQLAlchemy 2.0**	**2.0+**	**Async support, mature ORM**
ORM (TypeScript)	**Prisma**	**5+**	**Type-safe ORM, migrations, Studio**
GraphQL (NestJS)	**@nestjs/graphql**	**12+**	**Code-first GraphQL with decorators**
GraphQL (FastAPI)	**Strawberry**	**Latest**	**Type-safe GraphQL for Python**
WebSockets	**@nestjs/websockets**	**10+**	**Built-in WS gateway**
Job Queue	**Bull**	**4+**	**Redis-backed queue (NestJS)**
Task Scheduler	**@nestjs/schedule**	**4+**	**Cron jobs with decorators**
Email	**@nestjs-modules/mailer**	**Latest**	**Email service with templates**
Database	PostgreSQL	15+	ACID, JSON support, mature
Cache/Queue	Redis	7+	In-memory, pub/sub, sorted sets
File Storage	AWS S3	-	Scalable, cheap, reliable
CDN	CloudFront	-	Global distribution
Auth	JWT	-	Stateless, scalable
Password Hash	**Passlib (FastAPI)**	**Latest**	**Argon2id support**
Password Hash	**bcrypt (NestJS)**	**Latest**	**Industry standard**
Monitoring	Prometheus + Grafana	Latest	Metrics, dashboards, alerts
Logging (FastAPI)	**structlog**	**Latest**	**Structured JSON logging**
Logging (NestJS)	**@nestjs/common Logger**	**Built-in**	**Request tracking**
Testing (Python)	**pytest + httpx**	**Latest**	**Async test client**
Testing (NestJS)	**Jest**	**Built-in**	**Unit + E2E testing**
API Docs	**Auto-generated**	**-**	**FastAPI: Swagger UI, NestJS: GraphQL Playground**

FastAPI Backend Structure

```
backend/fastapi/
├── app/
│   ├── main.py                    # Application entry point
│   ├── config.py                  # Settings (Pydantic BaseSettings)
│   ├── dependencies.py            # Shared dependencies (DI)
│   │
│   ├── core/
│   │   ├── security.py            # JWT, password hashing, auth
│   │   ├── database.py            # SQLAlchemy async engine
│   │   ├── redis.py               # Redis connection
│   │   └── s3.py                  # S3 client
│   │
│   ├── models/                    # SQLAlchemy models
│   │   ├── user.py
│   │   ├── course.py
│   │   ├── note.py
│   │   ├── question.py
│   │   ├── answer.py
│   │   └── vote.py
│   │
│   ├── schemas/                   # Pydantic schemas (validation)
│   │   ├── user.py
│   │   ├── course.py
│   │   ├── note.py
│   │   ├── question.py
│   │   └── answer.py
│   │
│   ├── api/
│   │   ├── v1/
│   │   │   ├── __init__.py
│   │   │   ├── auth.py            # POST /auth/login, /register
│   │   │   ├── notes.py           # CRUD /notes
│   │   │   ├── questions.py       # CRUD /questions
│   │   │   ├── answers.py         # CRUD /answers
│   │   │   ├── votes.py           # POST /votes
│   │   │   ├── courses.py         # CRUD /courses
│   │   │   ├── tags.py            # CRUD /tags
│   │   │   ├── users.py           # User management
│   │   │   ├── upload.py          # File upload (presigned URLs)
│   │   │   └── admin.py           # Admin endpoints
│   │   └── deps.py                # API dependencies (auth, db)
│   │
│   ├── services/                  # Business logic layer
│   │   ├── auth_service.py
│   │   ├── note_service.py
│   │   ├── question_service.py
│   │   ├── vote_service.py
│   │   └── upload_service.py
│   │
│   └── utils/
│       ├── validators.py
│       ├── helpers.py
│       └── exceptions.py
│
├── tests/
│   ├── test_auth.py
│   ├── test_notes.py
│   ├── test_questions.py
```

```
│   └── conftest.py               # Pytest fixtures
│
├── alembic/                      # Database migrations
│   ├── versions/
│   └── env.py
│
├── requirements.txt
├── Dockerfile
└── .env
```

FastAPI Example Code

main.py:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware
from contextlib import asynccontextmanager

from app.core.database import engine
from app.models import Base
from app.api.v1 import auth, notes, questions, answers, votes, courses

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield
    # Shutdown
    await engine.dispose()

app = FastAPI(
    title="Academic Platform API",
    description="FastAPI backend for student collaboration",
    version="1.0.0",
    docs_url="/docs",
    redoc_url="/redoc",
    lifespan=lifespan
)

# Middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.add_middleware(GZipMiddleware, minimum_size=1000)

# Include routers
app.include_router(auth.router, prefix="/api/v1/auth", tags=["auth"])
app.include_router(notes.router, prefix="/api/v1/notes", tags=["notes"])
app.include_router(questions.router, prefix="/api/v1/questions", tags=["questions"])
app.include_router(answers.router, prefix="/api/v1/answers", tags=["answers"])
app.include_router(votes.router, prefix="/api/v1/votes", tags=["votes"])
app.include_router(courses.router, prefix="/api/v1/courses", tags=["courses"])

@app.get("/")
async def root():
    return {"message": "Academic Platform API", "version": "1.0.0"}

@app.get("/health")
async def health():
    return {"status": "healthy"}
```

models/user.py:

```
from sqlalchemy import Column, String, Integer, Boolean, DateTime, Enum
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.sql import func
import uuid
import enum

from app.core.database import Base

class UserRole(str, enum.Enum):
    USER = "user"
    MODERATOR = "moderator"
    ADMIN = "admin"

class User(Base):
    __tablename__ = "users"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    email = Column(String(255), unique=True, nullable=False, index=True)
    username = Column(String(50), unique=True, nullable=False, index=True)
    password_hash = Column(String(255), nullable=False)
    full_name = Column(String(100))
    avatar_url = Column(String(500))

    reputation_score = Column(Integer, default=0, index=True)
    role = Column(Enum(UserRole), default=UserRole.USER)
    is_verified = Column(Boolean, default=False)
    is_banned = Column(Boolean, default=False)

    created_at = Column(DateTime(timezone=True), server_default=func.now(), index=True)
    last_active = Column(DateTime(timezone=True), server_default=func.now())
```

schemas/user.py:

```
from pydantic import BaseModel, EmailStr, Field, ConfigDict
from datetime import datetime
from typing import Optional
from uuid import UUID

class UserBase(BaseModel):
    email: EmailStr
    username: str = Field(..., min_length=3, max_length=50, pattern="^[a-zA-Z0-9_]+$")
    full_name: Optional[str] = Field(None, max_length=100)

class UserCreate(UserBase):
    password: str = Field(..., min_length=8, max_length=100)

class UserLogin(BaseModel):
    email: EmailStr
    password: str

class UserResponse(UserBase):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    avatar_url: Optional[str] = None
    reputation_score: int
    role: str
    is_verified: bool
    created_at: datetime

class TokenResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "bearer"
    expires_in: int
    user: UserResponse
```

api/v1/auth.py:

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from sqlalchemy.ext.asyncio import AsyncSession
from datetime import timedelta

from app.core.database import get_db
from app.core.security import (
    verify_password, get_password_hash, create_access_token, create_refresh_token
)
from app.models.user import User
from app.schemas.user import UserCreate, UserLogin, UserResponse, TokenResponse
from app.services.auth_service import AuthService

router = APIRouter()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")

@router.post("/register", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
async def register(
    user_data: UserCreate,
    db: AsyncSession = Depends(get_db)
):
    """
```

```
    Register a new user account

    - **email**: Valid email address (preferably .edu)
    - **username**: Unique username (alphanumeric + underscore)
    - **password**: Minimum 8 characters
    - **full_name**: Optional display name
    """
    auth_service = AuthService(db)
    user = await auth_service.create_user(user_data)

    # TODO: Send verification email (enqueue job to NestJS)

    return user

@router.post("/login", response_model=TokenResponse)
async def login(
    credentials: UserLogin,
    db: AsyncSession = Depends(get_db)
):
    """
    Login with email and password

    Returns JWT access token (15min) and refresh token (7 days)
    """
    auth_service = AuthService(db)
    user = await auth_service.authenticate_user(credentials.email, credentials.password)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or password"
        )

    # Create tokens
    access_token = create_access_token(
        data={"sub": str(user.id), "role": user.role},
        expires_delta=timedelta(minutes=15)
    )
    refresh_token = create_refresh_token(
        data={"sub": str(user.id)},
        expires_delta=timedelta(days=7)
    )

    return TokenResponse(
        access_token=access_token,
        refresh_token=refresh_token,
        expires_in=900,  # 15 minutes
        user=UserResponse.model_validate(user)
    )

@router.get("/me", response_model=UserResponse)
async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db)
):
    """Get current authenticated user"""
    # Decode token and fetch user
    # Implementation in dependencies.py
    pass
```

core/security.py:

```
from datetime import datetime, timedelta
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext

SECRET_KEY = "your-secret-key-here"  # Load from env
ALGORITHM = "HS256"

pwd_context = CryptContext(schemes=["argon2"], deprecated="auto")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=15))
    to_encode.update({"exp": expire, "type": "access"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def create_refresh_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(days=7))
    to_encode.update({"exp": expire, "type": "refresh"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

NestJS Backend Structure

```
backend/nestjs/
├── src/
│   ├── main.ts                    # Application entry point
│   ├── app.module.ts              # Root module
│   │
│   ├── common/
│   │   ├── decorators/            # Custom decorators
│   │   ├── filters/               # Exception filters
│   │   ├── guards/                # Auth guards
│   │   ├── interceptors/          # Logging, transform
│   │   └── pipes/                 # Validation pipes
│   │
│   ├── config/
│   │   ├── config.module.ts
│   │   ├── database.config.ts
│   │   └── redis.config.ts
│   │
│   ├── database/
│   │   ├── database.module.ts
│   │   └── prisma.service.ts      # Prisma client
│   │
│   ├── graphql/                   # GraphQL setup
│   │   ├── graphql.module.ts
│   │   └── schema.gql             # Auto-generated schema
```

```
|   |
|   ├── modules/
|   |   ├── comments/
|   |   |   ├── comments.module.ts
|   |   |   ├── comments.service.ts
|   |   |   ├── comments.resolver.ts     # GraphQL resolver
|   |   |   ├── comments.gateway.ts      # WebSocket gateway
|   |   |   ├── dto/
|   |   |   |   ├── create-comment.dto.ts
|   |   |   |   └── update-comment.dto.ts
|   |   |   └── entities/
|   |   |       └── comment.entity.ts
|   |   |
|   |   ├── notifications/
|   |   |   ├── notifications.module.ts
|   |   |   ├── notifications.service.ts
|   |   |   ├── notifications.resolver.ts
|   |   |   ├── notifications.gateway.ts
|   |   |   └── dto/
|   |   |
|   |   ├── jobs/
|   |   |   ├── jobs.module.ts
|   |   |   ├── email/
|   |   |   |   ├── email.processor.ts
|   |   |   |   └── email.service.ts
|   |   |   ├── virus-scan/
|   |   |   |   └── virus-scan.processor.ts
|   |   |   └── analytics/
|   |   |       └── analytics.processor.ts
|   |   |
|   |   ├── leaderboard/
|   |   |   ├── leaderboard.module.ts
|   |   |   ├── leaderboard.service.ts
|   |   |   ├── leaderboard.resolver.ts
|   |   |   └── leaderboard.scheduler.ts
|   |   |
|   |   ├── cache/
|   |   |   ├── cache.module.ts
|   |   |   └── cache.service.ts
|   |   |
|   |   └── health/
|   |       └── health.controller.ts
|   |
|   └── utils/
|       ├── redis.ts
|       └── helpers.ts
|
├── prisma/
|   ├── schema.prisma              # Prisma schema
|   └── migrations/
|
├── test/
|   ├── app.e2e-spec.ts
|   └── jest-e2e.json
|
├── package.json
├── tsconfig.json
├── nest-cli.json
└── Dockerfile
```

NestJS Example Code

main.ts:

```
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // Global validation pipe
  app.useGlobalPipes(new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
  }));

  // CORS
  app.enableCors({
    origin: 'http://localhost:3000',
    credentials: true,
  });

  // Swagger documentation
  const config = new DocumentBuilder()
    .setTitle('Academic Platform GraphQL API')
    .setDescription('Real-time features and background jobs')
    .setVersion('1.0')
    .addBearerAuth()
    .build();
  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);

  await app.listen(3000);
  console.log(`🚀 NestJS server running on http://localhost:3000`);
  console.log(`📊 GraphQL Playground: http://localhost:3000/graphql`);
}
bootstrap();
```

app.module.ts:

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { GraphQLModule } from '@nestjs/graphql';
import { ApolloDriver, ApolloDriverConfig } from '@nestjs/apollo';
import { BullModule } from '@nestjs/bull';
import { ScheduleModule } from '@nestjs/schedule';

import { DatabaseModule } from './database/database.module';
import { CommentsModule } from './modules/comments/comments.module';
import { NotificationsModule } from './modules/notifications/notifications.module';
import { JobsModule } from './modules/jobs/jobs.module';
import { LeaderboardModule } from './modules/leaderboard/leaderboard.module';
import { CacheModule } from './modules/cache/cache.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),

    GraphQLModule.forRoot<ApolloDriverConfig>({
      driver: ApolloDriver,
      autoSchemaFile: 'src/graphql/schema.gql',
      sortSchema: true,
      playground: true,
      subscriptions: {
        'graphql-ws': true,
        'subscriptions-transport-ws': true,  // Legacy support
      },
      context: ({ req, connection }) => {
        if (connection) {
          return { req: connection.context };
        }
        return { req };
      },
    }),

    BullModule.forRoot({
      redis: {
        host: process.env.REDIS_HOST || 'localhost',
        port: parseInt(process.env.REDIS_PORT) || 6379,
      },
    }),

    ScheduleModule.forRoot(),
    DatabaseModule,
    CommentsModule,
    NotificationsModule,
    JobsModule,
    LeaderboardModule,
    CacheModule,
  ],
})
export class AppModule {}
```

modules/comments/comments.resolver.ts:

```
import { Resolver, Query, Mutation, Args, Subscription } from '@nestjs/graphql';
import { UseGuards } from '@nestjs/common';
import { PubSub } from 'graphql-subscriptions';
```

```
import { CommentsService } from './comments.service';
import { Comment } from './entities/comment.entity';
import { CreateCommentInput } from './dto/create-comment.input';
import { GqlAuthGuard } from '../../common/guards/gql-auth.guard';
import { CurrentUser } from '../../common/decorators/current-user.decorator';

const pubSub = new PubSub();

@Resolver(() => Comment)
export class CommentsResolver {
  constructor(private readonly commentsService: CommentsService) {}

  @Query(() => [Comment], { name: 'comments' })
  async findAll(
    @Args('targetId') targetId: string,
    @Args('targetType') targetType: string,
  ) {
    return this.commentsService.findAll(targetId, targetType);
  }

  @Mutation(() => Comment)
  @UseGuards(GqlAuthGuard)
  async createComment(
    @Args('input') input: CreateCommentInput,
    @CurrentUser() user: any,
  ) {
    const comment = await this.commentsService.create({
      ...input,
      authorId: user.id,
    });

    // Publish to subscribers
    pubSub.publish(`COMMENT_${input.targetType}_${input.targetId}`, {
      commentAdded: comment,
    });

    return comment;
  }

  @Subscription(() => Comment, {
    filter: (payload, variables) => {
      return (
        payload.commentAdded.targetId === variables.targetId &&
        payload.commentAdded.targetType === variables.targetType
      );
    },
  })
  commentAdded(
    @Args('targetId') targetId: string,
    @Args('targetType') targetType: string,
  ) {
    return pubSub.asyncIterator(`COMMENT_${targetType}_${targetId}`);
  }
}
```

modules/comments/entities/comment.entity.ts:

```
import { ObjectType, Field, ID, Int } from '@nestjs/graphql';

@ObjectType()
export class Comment {
  @Field(() => ID)
  id: string;

  @Field()
  content: string;

  @Field(() => Int)
  upvotes: number;

  @Field(() => Int)
  depth: number;

  @Field({ nullable: true })
  parentId?: string;

  @Field()
  authorId: string;

  @Field()
  targetId: string;

  @Field()
  targetType: string;

  @Field()
  createdAt: Date;

  @Field()
  updatedAt: Date;
}
```

modules/comments/dto/create-comment.input.ts:

```
import { InputType, Field } from '@nestjs/graphql';
import { IsString, IsUUID, IsOptional, MinLength, MaxLength } from 'class-validator';

@InputType()
export class CreateCommentInput {
  @Field()
  @IsString()
  @MinLength(1)
  @MaxLength(5000)
  content: string;

  @Field()
  @IsUUID()
  targetId: string;

  @Field()
  @IsString()
  targetType: string;

  @Field({ nullable: true })
  @IsOptional()
  @IsUUID()
  parentId?: string;
}
```

modules/jobs/email/email.processor.ts:

```
import { Process, Processor } from '@nestjs/bull';
import { Job } from 'bull';
import { MailerService } from '@nestjs-modules/mailer';

@Processor('email')
export class EmailProcessor {
  constructor(private readonly mailerService: MailerService) {}

  @Process('verification')
  async handleVerificationEmail(job: Job) {
    const { email, token, username } = job.data;

    await this.mailerService.sendMail({
      to: email,
      subject: 'Verify your email',
      template: 'verification',
      context: {
        username,
        verificationLink: `https://app.com/verify?token=${token}`,
      },
    });
  }

  @Process('password-reset')
  async handlePasswordReset(job: Job) {
    const { email, token, username } = job.data;

    await this.mailerService.sendMail({
      to: email,
      subject: 'Reset your password',
      template: 'password-reset',
      context: {
        username,
        resetLink: `https://app.com/reset-password?token=${token}`,
      },
    });
  }

  @Process('weekly-digest')
  async handleWeeklyDigest(job: Job) {
    const { email, username, popularNotes } = job.data;

    await this.mailerService.sendMail({
      to: email,
      subject: 'Your Weekly Study Digest',
      template: 'weekly-digest',
      context: {
        username,
        popularNotes,
      },
    });
  }
}
```

modules/leaderboard/leaderboard.scheduler.ts:

```
import { Injectable } from '@nestjs/common';
import { Cron, CronExpression } from '@nestjs/schedule';
import { InjectQueue } from '@nestjs/bull';
import { Queue } from 'bull';

import { LeaderboardService } from './leaderboard.service';

@Injectable()
export class LeaderboardScheduler {
  constructor(
    private readonly leaderboardService: LeaderboardService,
    @InjectQueue('email') private emailQueue: Queue,
  ) {}

  @Cron('59 23 * * 0')  // Every Sunday at 11:59 PM
  async resetWeeklyLeaderboard() {
    console.log('🏆 Resetting weekly leaderboard...');

    // Get top 10 winners
    const winners = await this.leaderboardService.getTop(10);

    // Archive winners
    await this.leaderboardService.archiveWinners(winners);

    // Send congratulations emails
    for (const [index, winner] of winners.entries()) {
      await this.emailQueue.add('leaderboard-winner', {
        email: winner.email,
        username: winner.username,
        rank: index + 1,
        score: winner.score,
      });
    }

    // Reset weekly scores
    await this.leaderboardService.resetWeekly();

    console.log('✅ Leaderboard reset complete');
  }

  @Cron(CronExpression.EVERY_6_HOURS)
  async warmCache() {
    console.log('🔥 Warming cache...');
    await this.leaderboardService.warmCache();
  }
}
```

prisma/schema.prisma:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

```
model User {
  id              String    @id @default(uuid())
  email           String    @unique
  username        String    @unique
  passwordHash    String    @map("password_hash")
  fullName        String?   @map("full_name")
  avatarUrl       String?   @map("avatar_url")
  reputationScore Int       @default(0) @map("reputation_score")
  role            Role      @default(USER)
  isVerified      Boolean   @default(false) @map("is_verified")
  isBanned        Boolean   @default(false) @map("is_banned")
  createdAt       DateTime  @default(now()) @map("created_at")
  lastActive      DateTime  @default(now()) @map("last_active")

  comments        Comment[]
  notifications   Notification[]

  @@map("users")
}

enum Role {
  USER
  MODERATOR
  ADMIN
}

model Comment {
  id         String   @id @default(uuid())
  content    String
  upvotes    Int      @default(0)
  depth      Int      @default(0)
  authorId   String   @map("author_id")
  targetId   String   @map("target_id")
  targetType String   @map("target_type")
  parentId   String?  @map("parent_id")
  isDeleted  Boolean  @default(false) @map("is_deleted")
  createdAt  DateTime @default(now()) @map("created_at")
  updatedAt  DateTime @updatedAt @map("updated_at")

  author     User     @relation(fields: [authorId], references: [id], onDelete: Cascade)

  @@map("comments")
}

model Notification {
  id             String   @id @default(uuid())
  userId         String   @map("user_id")
  type           String
  title          String
  message        String
  actionUrl      String?  @map("action_url")
  referenceId    String?  @map("reference_id")
  referenceType String?  @map("reference_type")
  isRead         Boolean  @default(false) @map("is_read")
  readAt         DateTime? @map("read_at")
  createdAt      DateTime @default(now()) @map("created_at")

  user           User     @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```
  @@map("notifications")
}
```

Updated Developer Workload (Still 50/50!)

FastAPI Developer Responsibilities (50%)

Lines of Code: ~5,000-6,000
Complexity: High

1. **Authentication & Authorization (20%)**
2. User registration, login (Pydantic validation)
3. JWT token management
4. Password hashing (Argon2id via Passlib)
5. Email verification coordination
6. RBAC middleware
7. **Content Management (30%)**
8. Notes CRUD (async SQLAlchemy)
9. Questions CRUD
10. Answers CRUD
11. Search (PostgreSQL full-text)
12. File upload coordination (S3 presigned URLs)
13. **Voting & Reputation (10%)**
14. Vote endpoints
15. Reputation calculation
16. Vote validation
17. **Course & Tags (10%)**
18. Course CRUD
19. Tag CRUD
20. Associations
21. **User Management (10%)**
22. User profiles
23. Statistics
24. Bookmarks
25. **Admin & Moderation (10%)**
26. Reports
27. Banning
28. Moderation queue
29. **File Coordination (10%)**
30. S3 presigned URLs
31. Upload validation
32. File metadata

NestJS Developer Responsibilities (50%)

Lines of Code: ~5,000-6,000

Complexity: High

1. **Real-time Communication (25%)**
2. GraphQL subscriptions
3. WebSocket gateways
4. Real-time comments
5. Real-time votes
6. Connection management
7. **Comments System (10%)**
8. Comment CRUD (GraphQL)
9. Threading
10. Real-time broadcasting
11. **Notification System (15%)**
12. In-app notifications
13. Email queue
14. Push notifications (future)
15. Notification delivery
16. **Background Jobs (20%)**
17. Bull queues
18. Email processing
19. Virus scanning
20. Analytics
21. File processing
22. **Scheduled Tasks (10%)**
23. Cron jobs (@nestjs/schedule)
24. Leaderboard reset
25. Daily digests
26. Cache warming
27. **Leaderboard System (10%)**
28. Redis sorted sets
29. Real-time updates
30. Badge system
31. Achievement tracking
32. **Caching (5%)**
33. Redis cache service
34. Cache invalidation
35. Rate limiting
36. **Search Coordination (5%)**
37. Search indexing jobs
38. Suggestions

Why This Stack is Better

FastAPI Advantages Over Flask

✅ **3x faster** - ASGI vs WSGI
✅ **Auto documentation** - Swagger UI out-of-box
✅ **Type safety** - Pydantic validation at runtime
✅ **Async native** - Better for I/O operations
✅ **Modern Python** - Uses latest features
✅ **Better DX** - Auto-completion everywhere

NestJS Advantages Over Plain Node.js

✅ **Enterprise architecture** - Built-in DI, modules
✅ **TypeScript first** - Type safety everywhere
✅ **Less boilerplate** - Decorators for everything
✅ **Built-in testing** - Jest integration
✅ **GraphQL native** - Code-first approach
✅ **WebSockets native** - @nestjs/websockets
✅ **Microservices ready** - Easy to split later
✅ **Auto documentation** - Swagger + GraphQL Playground

Development Timeline (Same Balance!)

The timeline remains the same - both developers still work equally across all phases. The only difference is they're using better tools:

Phase 1 (Weeks 1-6): - FastAPI dev: Auth, Notes, Questions, Votes (~2,500 LOC) - NestJS dev: WebSockets, Comments, Jobs, Emails (~2,500 LOC)

Phase 2 (Weeks 7-10): - FastAPI dev: Questions advanced, Answers (~1,500 LOC) - NestJS dev: Real-time Q&A, Notifications (~1,500 LOC)

Phase 3 (Weeks 11-16): - FastAPI dev: Admin, Moderation, Advanced search (~1,500 LOC) - NestJS dev: Leaderboards, Analytics, Performance (~1,500 LOC)

Total: Still 50/50 split! ✅

Deployment (Same Strategy)

Docker, AWS ECS, PostgreSQL RDS, Redis ElastiCache - all remains the same. The container images are just different:

```
# FastAPI Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

# NestJS Dockerfile
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
CMD ["node", "dist/main"]
```

Conclusion

This is the ULTIMATE stack for 2025+:

✅ **FastAPI** - Modern, fast, type-safe Python
✅ **NestJS** - Enterprise TypeScript framework
✅ **Still 50/50 balanced** - Equal workload
✅ **Better DX** - Auto-completion, auto-docs
✅ **Production ready** - Battle-tested frameworks
✅ **Future-proof** - Modern best practices

Ready to build! 🚀